price lookup. Instead create a StockExchange interface and inject it into Portfolio, so tests can used a fixed stub. This follows the Dependency Inversion Principle (DIP): classes should depend on abstractions, not on concrete details.

# GOOD PRACTICES AND CONCEPTUAL MODELING

## Conceptual Modeling: What and Why

Conceptual modeling means capturing ideas and concepts in a form that supports reasoning, analysis, prioritization, and design. Its purposes are to:

- understand and visualize a real-world systems as it is today.
- envision how system should become.
- document key decisions and specify a solution.
- plan the construction
- communicate efficiently, which the slides stress most.

## Conceptual Modeling: When

Modeling can be done at any point in development, starting at the very beginning, and from several perspectives: analysis, design, specification, implementation, and deployment.

## Conceptual Modeling with UML

UML (Unified Modeling Language) is a family modeling languages. The main diagram types are:

- Use case diagrams, wich show who needs the system and what their goals are.
- Class diagrams, wich show how objects communicate with each other.
- State charts, wich show the internal behavior of the objects of a class or of a method.

## Organize Software Into Loosely Coupled Layers

A system should be designed in layers. Each layer offers a well-defined abstraction to the layers above it. A layer can use components from its own layer and the abstractions provided by lower layers. The examples architecture, from top to bottom, is User Interface, Application Logic, Persistence and Communications (side by side), and Operating System.

## Prefer Aggregation Over Inheritance

In class-based languages there are two basic ways to reuse code.

Escaneado con CamScanner

- Aggregation: Object A contains object B and reuse it.
- Inheritance: class A inherits from class B, so A objects are B objects and have all of B's capabilities.

Aggregation relationships can change at runtime so they are more flexible. When other considerations are equal, choose aggregation. Composition is a stronger form of aggregation.

## Program to an Interface or Abstraction

Programming to an interface or abstraction manages coupling, removes accidental complexity, and eliminates unnecessary dependencies. A class interface is one type of abstraction, and it consist of all of the class's public methods and properties.

## Use Identifiers that Improve Readability and Maintainability

Meaningful names within their scope make code easir to read and maintain

- Classes represent sets of objects and named after a typical instance (Widget, a class holding sets of widgets would be Widgets). Names should express the object's purpose or intended use. They use lower CamelCase (myWidget).
- Methods perform actions, so their names should be verbs of verb phrases (Display, computeCost).

## Testing with Executable Unit Test Cases

Testing individual units of code with executable tests.

- helps find design errors
- verifies correctness through repeatable tests
- documents how a method or class is used
- enables regression testing.

# INTRODUCTION to UML 1.05 07

Introduces the Unified Modeling Language (UML) as an expressive, semi-formal, and process-independent object-oriented modeling language. It explains that UML utilizes various modeling elements - categorized as structural, behavioral, behavioral organizational, and annotational - to construct different types of structural and behavioral views, such as logical or implementation views. Software engineers can visualize, specify, construct, and document complex software systems from multiple perspectives, including analysis, specification, and implementation.

Escaneado con CamScanner

DD MM AA

## Basic Use Cases 1.08

The section focuses on capturing user-visible system functionality through the use of actors and use cases. Actors represent external entities such as users or other software systems that interact with the application to achieve specific goals. Use cases document these discrete goals independently of software design choices. Furthermore, that document details how to visually map these interactions Use Case Diagrams, explaining key relationships such as <<extend>> for variations, <<include>> for required sub-goals, and generalizations to represent specialized subsets of behavior.

## Class Diagrams 1.05 09

The final document explores the fundamental building blocks of object-oriented systems. objects, wich encapsulate state and behavior and classes, wich serve as abstractions for collections of similar objects. It breaks down the syntax of a class, highlighting its name, attributes, and operations. Crucially, it explains how class diagrams visually model the static structure of a system by illustrating the types of objects and their structural relationships, including associations, generalizations, dependencies, aggregations, and compositions, alongside strict multiplicity constraints.

Escaneado con CamScanner